Building ETL/ELT Pipelines

Batch, Streaming, Kafka & Apache Spark

Session 3 of 7 | 4 Hours

Big Data Engineering Programme
Academic Year 2025–2026

Session Overview

Duration	4 hours (theory + lab)
Theory	ETL vs ELT, Batch vs Streaming, Spark architecture
Lab	Design a Kafka + Spark Structured Streaming ETL pipeline
Prerequisites	Sessions 1–2 completed; Docker cluster running
Tools used	Python 3, PySpark, kafka-python-ng, Docker Compose

Learning Objectives

By the end of this session, students will be able to:

- Distinguish ETL from ELT and choose the right pattern for a given use case.
- Compare batch and streaming ETL along six key dimensions.
- Explain Apache Spark's architecture: driver, executors, DAG scheduler, and lazy evaluation.
- Write a PySpark Structured Streaming job that reads from Kafka, transforms data, and writes to a sink.
- Apply windowed aggregations and watermarks to handle late-arriving events.
- Integrate Kafka as a Spark streaming source and write enriched data to a Parquet sink.

Contents

1. Part I – ETL vs ELT: Concepts and Patterns

1.1 What Is a Data Pipeline?

Definition – Data Pipeline

A **data pipeline** is a sequence of automated steps that move data from one or more *sources* to one or more *destinations*, applying *transformations* along the way to make the data usable for downstream consumers (analysts, ML models, dashboards, APIs).

Every data pipeline answers three questions:

- **Where does the data come from?** (source: database, API, Kafka, files)
- **What do we do to it?** (transform: clean, join, aggregate, enrich)
- **Where does it go?** (sink: data warehouse, data lake, another Kafka topic)

1.2 ETL – Extract, Transform, Load

Definition – ETL

ETL (Extract, Transform, Load) is the traditional pipeline pattern where data is first *extracted* from sources, *transformed* in a dedicated processing layer (cleaned, normalised, joined), and only then *loaded* into the destination store in its final, analysis-ready form.

Figure 1: ETL pattern: transform first, then load into the destination.

When to use ETL:

- Destination has limited compute (traditional RDBMS, small DW).
- Data must be clean and validated before touching production storage.
- Regulatory requirements mandate transformation before persistence.
- Legacy systems: pre-cloud era, most ETL tools (Informatica, SSIS).

1.3 ELT – Extract, Load, Transform

Definition – ELT

ELT (Extract, Load, Transform) reverses the order: raw data is loaded immediately into a powerful storage layer (cloud data warehouse or data lake), and transformations happen *inside* the destination using its native compute.

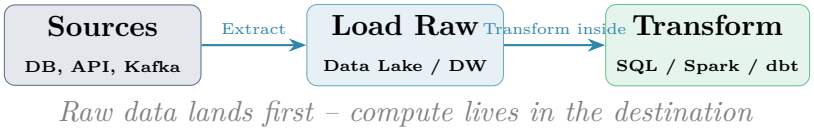

Figure 2: ELT pattern: load raw first, transform using destination compute.

When to use ELT:

- Destination has powerful compute (BigQuery, Snowflake, Redshift, Spark).
- Raw data must be preserved for audit, replay, or future use.
- Transformation logic evolves frequently (easier to re-run in-place).
- Modern cloud-native stacks with tools like *dbt*.

1.4 ETL vs ELT: Direct Comparison

Table 1: ETL vs ELT comparison across key dimensions

lightbg Dimension	**ETL**	**ELT**
Transform location	External processing layer	Inside the destination
Raw data preserved?	No – only final form stored	Yes – raw always available
Compute requirements	Dedicated ETL server	Powerful destination (cloud DW)
Schema at write time	Schema-on-write (enforced)	Schema-on-read (flexible)
Time to first load	Slower (transform first)	Faster (load raw immediately)
Re-processing	Must re-extract from source	Re-run transforms on existing raw
Typical tools	SSIS, Informatica, Talend	dbt, Spark, BigQuery SQL
Cost model	Fixed ETL server cost	Pay-per-query compute

2. Part II – Batch vs Streaming ETL

2.1 Batch Processing

Definition – Batch Processing

Batch processing operates on a *bounded*, finite dataset collected over a fixed time window (e.g., all orders from yesterday). The job runs to completion, producing an output, then stops. Data is processed in large chunks at scheduled intervals.

Characteristics:

- High throughput – optimised for moving large volumes efficiently.
- High latency – results available minutes to hours after events occur.
- Simple failure model – on failure, re-run the entire job.
- Simpler to reason about – bounded datasets, no state management.
- Typical tools: Apache Spark (batch mode), Hive, dbt, Hadoop MapReduce.

2.2 Streaming Processing

Definition – Streaming Processing

Streaming processing operates on an *unbounded*, continuous data stream. Records are processed individually or in micro-batches as they arrive, producing results continuously with low latency.

Characteristics:

- Low latency – results available milliseconds to seconds after events.
- Lower throughput per second compared to batch.
- Complex failure model – must handle partial state, out-of-order events.
- Stateful processing – windowing, joins require maintaining state.
- Typical tools: Spark Structured Streaming, Apache Flink, Kafka Streams.

2.3 Batch vs Streaming: Six-Dimension Comparison

Table 2: Batch vs Streaming ETL along six key dimensions

lightbg Dimension	Batch	Streaming
Data model	Bounded (finite window)	Unbounded (continuous)
Latency	Minutes to hours	Milliseconds to seconds
Throughput	Very high (TB/job)	Lower per second
State management	Stateless (per-run)	Stateful (windows, joins)
Failure handling	Re-run entire job	Checkpoint + resume
Typical use cases	Nightly reports, historical ETL	Fraud detection, monitoring

2.4 Lambda and Kappa Architectures

Real-world systems often combine both paradigms.

2.4.1 Lambda Architecture

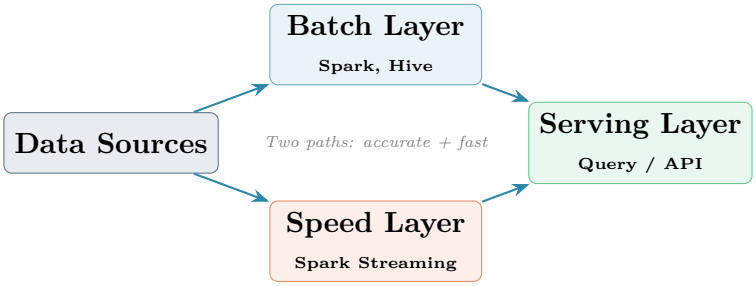

Figure 3: Lambda architecture: batch layer for accuracy, speed layer for low latency.

2.4.2 Kappa Architecture

The Kappa architecture simplifies Lambda by eliminating the batch layer: *everything* is a stream, including historical reprocessing (by replaying Kafka from offset 0). This is the approach taken by Kafka + Spark Structured Streaming.

Kappa in Practice

Kafka's configurable retention makes historical reprocessing trivial: set `auto.offset.reset=earliest` and replay the entire topic. The streaming job produces the same results as a hypothetical batch job would – with no separate batch codebase to maintain.

3. Part III – Apache Spark Architecture

3.1 What Is Apache Spark?

Definition – Apache Spark

Apache Spark is an open-source, distributed **unified analytics engine** for large-scale data processing. It provides a unified API for batch processing, streaming, SQL, machine learning, and graph computation – all on the same cluster, using the same programming model.

Spark's key differentiator from Hadoop MapReduce is **in-memory processing**: intermediate results are kept in RAM rather than written to disk between stages, making Spark 10–100× faster for iterative workloads.

3.2 Spark Cluster Architecture

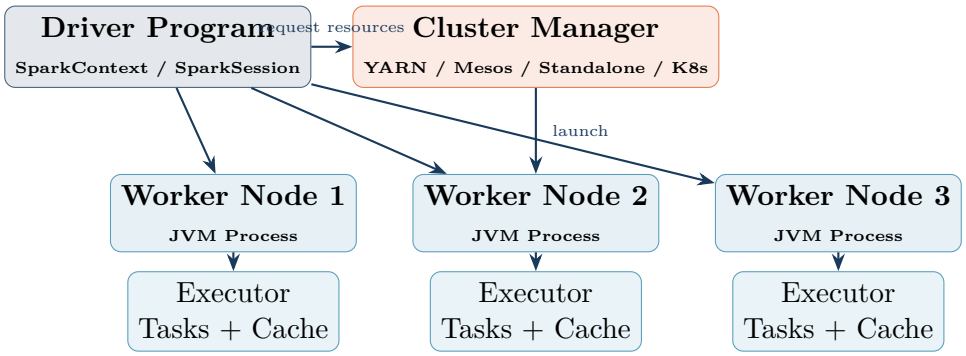

Figure 4: Spark cluster: the Driver orchestrates; Executors do the work.

3.3 Core Concepts

3.3.1 Driver vs Executor

lightbg Component	Role	Key facts
Driver	Runs the user's `main()` function; builds the DAG; schedules tasks; coordinates executors	One per application; bottleneck if too much data collected here
Executor	Runs tasks assigned by the Driver; caches RDD/-DataFrame partitions in memory; reports results	Many per application; scale horizontally; isolated JVM processes

3.3.2 RDD, DataFrame, and Dataset

Table 3: Spark's three data abstractions

lightbg Abstraction	Language	When to use
RDD (Resilient Distributed Dataset)	Scala, Java, Python	Low-level control; unstructured data; custom serialisation
DataFrame	Python, Scala, Java, R	Structured/semi-structured data; SQL-like operations; Catalyst optimiser
Dataset	Scala, Java only	Type-safe API + Catalyst optimiser; best of both worlds

Use DataFrames for ETL

For ETL and streaming pipelines, always use the **DataFrame / Dataset API** (not RDDs). The **Catalyst query optimiser** and **Tungsten** execution engine provide automatic query planning, predicate pushdown, and vectorised execution that make DataFrames dramatically faster than equivalent RDD code.

3.3.3 Lazy Evaluation and the DAG

Spark uses **lazy evaluation**: transformations (`filter`, `select`, `join`) are not executed immediately. Instead, Spark builds a *Directed Acyclic Graph (DAG)* of transformations and only executes when an *action* is called (`show`, `count`, `write`).

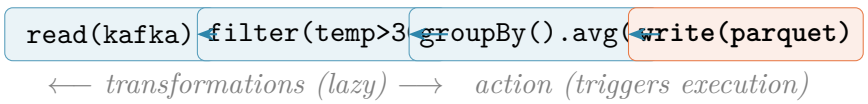

Figure 5: Spark DAG: transformations are planned lazily; the action triggers the physical execution.

3.4 Spark Structured Streaming

Definition – Structured Streaming

Spark Structured Streaming is the stream processing engine built on top of the Spark SQL engine. It models a live data stream as an *unbounded table* that is continuously appended. Users write the same DataFrame/SQL API as batch jobs, and Spark handles the complexity of incremental execution automatically.

3.4.1 The Unbounded Table Model

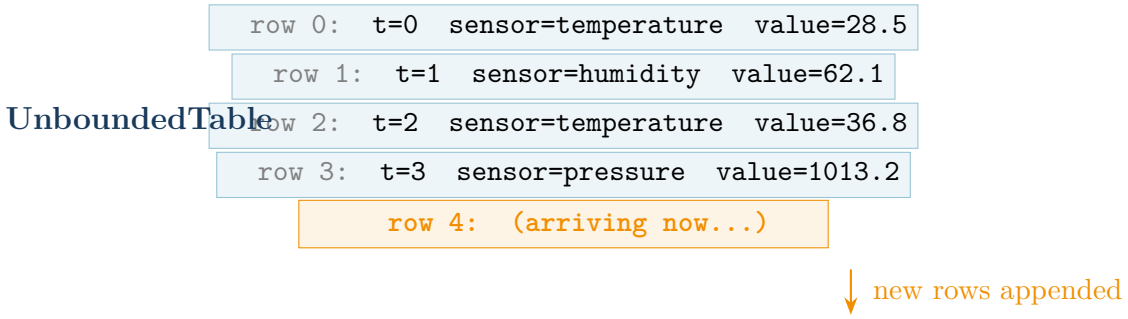

Figure 6: Structured Streaming treats the stream as a table that grows forever.

3.4.2 Output Modes

Structured Streaming supports three output modes that control what is written to the sink at each trigger:

Table 4: Structured Streaming output modes

lightbg Mode	What is written	Use case
`append`	Only new rows added since last trigger	Append-only sinks (Parquet, Kafka, S3)
`update`	Only rows that changed since last trigger	Key-value sinks (databases, Redis)
`complete`	The entire updated result table	Small aggregation result sets

3.4.3 Watermarks and Late Data

In streaming, events can arrive *late* (e.g., a mobile sensor that was offline). Without handling late data, aggregation windows would never close.

Definition – Watermark

A **watermark** is Spark's estimate of the *maximum event-time lag* it will tolerate. Events arriving later than the watermark threshold are dropped. The watermark is defined as:

$$\text{watermark} = \max(\text{event time seen so far}) - \text{threshold}$$

This allows Spark to safely expire old state and close time windows.

```
# Watermark: tolerate events up to 10 minutes late
df.withWatermark("event_time", "10 minutes") \
  .groupBy(window("event_time", "5 minutes"), "sensor") \
  .agg(avg("value").alias("avg_value"))
```

window("event_time", "5 minutes") groups records into non-overlapping 5-minute time buckets based on the event's own timestamp.
withWatermark("event_time", "10 minutes") tells Spark: discard any record whose event_time is more than 10 minutes behind the latest event time seen. Without this, Spark must keep state for all past windows forever.

4. Part IV – Kafka + Spark Integration

4.1 Why Kafka + Spark?

Kafka and Spark are the two most widely deployed components of a real-time data platform, and they are designed to work together:

lightbg Component	Role in the pipeline
Apache Kafka	Ingest, buffer, and transport events at high throughput. Acts as the *source* for Spark and the *sink* for processed results.
Apache Spark	Read events from Kafka, apply transformations, aggregations, and ML models. Write results to Parquet, databases, or back to Kafka.

4.2 Reading from Kafka in Spark

Spark reads Kafka topics using the `kafka` format. Each Kafka record becomes one row in a Spark DataFrame with the following schema:

Table 5: Schema of a Kafka record read by Spark

lightbg Column	Type	Content
`key`	`binary`	Message key (bytes)
`value`	`binary`	Message value (bytes) – your payload
`topic`	`string`	Topic name
`partition`	`int`	Partition number
`offset`	`long`	Offset within partition
`timestamp`	`timestamp`	Producer or broker timestamp
`timestampType`	`int`	0 = CreateTime, 1 = LogAppendTime

⚠ Common Pitfall

The **value** column arrives as raw **binary bytes**, not as a Python string or dictionary. You *must* cast it to string and then parse it with `from_json()` before you can access individual fields. Forgetting this step is the most common beginner mistake.

4.3 The ETL Pipeline Architecture for This Session

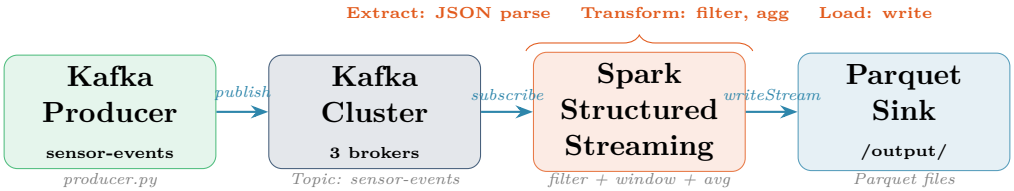

Figure 7: Session 3 ETL pipeline: Kafka (source) → Spark Streaming (process) → Parquet (sink).

5. Part V – Lab Session

Lab 3 – Kafka + Spark Structured Streaming ETL Pipeline

Duration	90 minutes
Objective	Build a PySpark streaming job that reads sensor events from Kafka, filters anomalies,
Tools	Python 3, PySpark 3.5, kafka-python-ng, Docker Compose
Difficulty	●●● Intermediate

Lab Timeline

Time	Activity	Goal
0–10 min	Setup: cluster + dependencies	Verify environment
10–20 min	Step 1: Spark reads raw Kafka stream	Understand binary schema
20–35 min	Step 2: Parse JSON payload	Cast + from_json
35–50 min	Step 3: Filter and alert	Apply business logic
50–65 min	Step 4: Windowed aggregation	groupBy + window + avg
65–80 min	Step 5: Write to Parquet sink	Checkpointing + triggers
80–90 min	Inspect output + Reflection	Read Parquet, answer questions

5.1 Step 0 – Environment Setup

5.1.1 0a – Start the Kafka Cluster

```
cd kafka-lab
docker compose up -d
docker compose ps     # all 4 containers: Up
```

The same 3-broker cluster from Sessions 1 and 2 is reused. Make sure the `sensor-events` topic exists: `docker exec kafka1 kafka-topics -bootstrap-server kafka1:29092 -list`

5.1.2 0b – Install Python Dependencies

```
# Activate virtual environment from TP2 (or create a new one)
source venv/bin/activate

pip install pyspark==3.5.3 kafka-python-ng
```

PySpark 3.5 includes the Kafka connector (`spark-sql-kafka`) as an optional package that must be specified at runtime via `-packages`. No separate JAR download is needed – Spark fetches it from Maven Central automatically on first run.

5.1.3 0c – Create Output Directory

```
mkdir -p /tmp/spark-etl/output
mkdir -p /tmp/spark-etl/checkpoint
```

output/ is where Spark writes Parquet files. **checkpoint/** is where Spark saves its streaming state (offsets, aggregation state) to recover from failures. Without a checkpoint directory, a restarted Spark job cannot resume – it would reprocess from the beginning.

5.2 Step 1 – Create the Spark Session

Create a file `etl_pipeline.py`. We build it section by section.

5.2.1 1a – Imports

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, from_json, avg, window,
    to_timestamp, expr, lit
)
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType, LongType
)
```

from_json converts a JSON string column into a Spark struct (the key function for parsing Kafka payloads).
window creates time-based buckets for aggregation (e.g., 5-minute windows).
StructType / StructField define the expected JSON schema so Spark can validate and type-cast fields automatically.

5.2.2 1b – Build the SparkSession

```
spark = SparkSession.builder \
    .appName("SessionETLPipeline") \
    .master("local[*]") \
    .config(
        "spark.jars.packages",
        "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3"
    ) \
    .config("spark.sql.shuffle.partitions", "3") \
    .config("spark.streaming.stopGracefullyOnShutdown", "true") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
```

.master("local[*]") runs Spark locally using all available CPU cores. In production, this would be replaced by a YARN or Kubernetes URL.
spark.jars.packages tells Spark to download the Kafka connector JAR automatically. The format is `groupId:artifactId:version`. The suffix `2.12` refers to the Scala version (not Spark version).
spark.sql.shuffle.partitions=3 reduces the default 200 shuffle partitions to 3, which is appropriate for a 3-partition Kafka topic on a local machine. The default 200 would create many empty files.
stopGracefullyOnShutdown=true allows Spark to finish the current micro-batch before stopping when it receives a SIGTERM.

5.3 Step 2 – Read the Raw Kafka Stream

```
KAFKA_BROKERS = "localhost:9092,localhost:9094,localhost:9096"
TOPIC = "sensor-events"

raw_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
    .option("subscribe", TOPIC) \
    .option("startingOffsets", "earliest") \
    .option("failOnDataLoss", "false") \
    .load()
```

.format("kafka") activates the Kafka source connector loaded from the JAR specified in `spark.jars.packages`.
subscribe accepts a single topic. Use `subscribePattern` for regex (e.g., `"sensor-.*"` to read all sensor topics at once).
startingOffsets="earliest" tells Spark to read from offset 0 the first time it starts. On subsequent runs with a checkpoint directory, Spark ignores this option and resumes from the last committed offset.
failOnDataLoss="false" prevents the job from crashing if Kafka offsets are no longer available (e.g., topic was deleted). In production, set to `"true"` to catch data gaps early.

Print the schema to understand the raw structure:

```
raw_stream.printSchema()
# root
#  |-- key: binary (nullable = true)
#  |-- value: binary (nullable = true)
#  |-- topic: string (nullable = true)
#  |-- partition: integer (nullable = true)
#  |-- offset: long (nullable = true)
#  |-- timestamp: timestamp (nullable = true)
#  |-- timestampType: integer (nullable = true)
```

5.4 Step 3 – Parse the JSON Payload

5.4.1 3a – Define the Expected Schema

```
sensor_schema = StructType([
    StructField("sensor",    StringType(),  nullable=False),
    StructField("value",     DoubleType(),  nullable=False),
    StructField("unit",      StringType(),  nullable=True),
    StructField("timestamp", LongType(),    nullable=False),
    StructField("source",    StringType(),  nullable=True),
])
```

Defining the schema explicitly is **always preferred** over `schema inference` for streaming jobs because:

- Schema inference requires reading all data first (impossible for an unbounded stream).
- Explicit schema enables Spark's **Catalyst** optimiser to plan the query without scanning data.
- Type mismatches (e.g., value arriving as string instead of double) produce `null` rather than crashing – you can detect them.

5.4.2 3b – Cast and Parse

```
parsed_stream = raw_stream \
    .select(
        # Cast binary key to string
        col("key").cast("string").alias("message_key"),
        # Cast binary value to string, then parse JSON
        from_json(
            col("value").cast("string"),
            sensor_schema
        ).alias("data"),
        # Keep Kafka metadata
        col("partition"),
        col("offset"),
        col("timestamp").alias("kafka_timestamp"),
    ) \
    .select(
        col("message_key"),
        col("data.sensor"),
        col("data.value"),
        col("data.unit"),
        # Convert epoch ms to Spark timestamp for windowing
        to_timestamp(
            expr("data.timestamp / 1000")
        ).alias("event_time"),
        col("partition"),
        col("offset"),
    )
```

Two-step select pattern: the first `select` creates the `data` struct; the second `select` flattens it into individual columns. This is cleaner than deeply nesting `col("data.sensor")` inside the first select.
to_timestamp(expr("data.timestamp / 1000")): the producer stored the timestamp as epoch *milliseconds* (a long integer), but Spark's `window()` function requires a `timestamp` column in *seconds*. Dividing by 1000 and casting converts correctly.
If `from_json` encounters a malformed record, it returns `null` for the `data` struct. Add `.filter(col("data").isNotNull())` after parsing to discard bad records and route them to a dead-letter topic in production.

5.5 Step 4 – Apply Business Logic: Filter and Alert

```
#         4a: Filter out null records (parse failures)

clean_stream = parsed_stream \
    .filter(col("sensor").isNotNull()) \
    .filter(col("value").isNotNull())

#         4b: Flag anomalies

TEMP_THRESHOLD  = 35.0
HUM_THRESHOLD   = 90.0

flagged_stream = clean_stream.withColumn(
    "is_anomaly",
    (
        ((col("sensor") == "temperature") &
         (col("value") > TEMP_THRESHOLD))
        |
        ((col("sensor") == "humidity") &
         (col("value") > HUM_THRESHOLD))
    )
)
```

.filter() on a streaming DataFrame is applied *per micro-batch* as records arrive – exactly the same syntax as batch filtering. This is the power of Structured Streaming's unified API.
withColumn("is_anomaly", ...) adds a boolean column without materialising anything – it is just another node in the DAG that will be evaluated when a write action triggers execution.
In production, anomalous records would be written to a separate Kafka topic for alerting (e.g., `sensor-alerts`), while the full stream continues to the main sink.

5.6 Step 5 – Windowed Aggregation

```
#         5a: Add watermark to handle late data

watermarked = flagged_stream \
    .withWatermark("event_time", "2 minutes")
```

```
#         5b: Group by 5-minute window and sensor type

windowed_avg = watermarked \
    .groupBy(
        window(col("event_time"), "5 minutes"),
        col("sensor"),
    ) \
    .agg(
        avg(col("value")).alias("avg_value"),
        avg(col("value").cast("double")).alias("avg_val"),
    )
```

withWatermark("event_time", "2 minutes"): Spark will tolerate events arriving up to 2 minutes late. An event more than 2 minutes behind the latest seen event_time is silently dropped. This is the mechanism that allows Spark to finalize a time window and free the associated memory.
window(col("event_time"), "5 minutes"): divides the event_time axis into 5-minute non-overlapping buckets. The `window` column in the result is a struct with two fields: `window.start` and `window.end` (both timestamps).
Why not use Kafka timestamp? The event's own timestamp (`event_time`) reflects *when the event happened* in the real world. Using the Kafka ingestion timestamp would group events by *when they arrived*, which may be affected by network delays. Event-time processing produces semantically correct results.

5.7 Step 6 – Write to Parquet Sink

5.7.1 6a – Write the Aggregated Stream

```
OUTPUT_PATH     = "/tmp/spark-etl/output"
CHECKPOINT_PATH = "/tmp/spark-etl/checkpoint"

query = windowed_avg \
    .writeStream \
    .outputMode("update") \
    .format("parquet") \
    .option("path", OUTPUT_PATH) \
    .option("checkpointLocation", CHECKPOINT_PATH) \
    .trigger(processingTime="10 seconds") \
    .start()

print("Streaming query started. Press Ctrl+C to stop.")
query.awaitTermination()
```

.outputMode("update"): only write rows whose aggregated value changed in this micro-batch. This is the correct mode for streaming aggregations because `append` mode is not supported when using watermarks without a complete guarantee.
.format("parquet"): Spark writes columnar Parquet files, the standard format for data lakes (covered in Session 4). Each micro-batch creates a new set of files partitioned by the trigger interval.
.option("checkpointLocation", ...): the checkpoint directory stores:

- The Kafka offsets read so far (prevents reprocessing on restart).
- The watermark state (prevents re-opening closed windows).
- The aggregation state (running averages per window+sensor).

Without a checkpoint, every restart reprocesses from `earliest`.
.trigger(processingTime="10 seconds"): Spark runs a new micro-batch every 10 seconds. Reduce for lower latency; increase for higher throughput. Use `trigger(once=True)` to process all available data and stop (batch-mode streaming, useful for backfill).

5.7.2 6b – Write the Raw Flagged Stream (Optional)

You can run a *second* streaming query on the same DataFrame to write the raw (un-aggregated) records including the anomaly flag:

```
RAW_OUTPUT     = "/tmp/spark-etl/raw"
RAW_CHECKPOINT = "/tmp/spark-etl/checkpoint-raw"

query_raw = flagged_stream \
    .writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path", RAW_OUTPUT) \
    .option("checkpointLocation", RAW_CHECKPOINT) \
    .trigger(processingTime="10 seconds") \
    .start()
```

Spark supports **multiple simultaneous streaming queries** on the same SparkSession. Each query has its own checkpoint, offset tracking, and output sink. This is how you implement the *dual-write* pattern: raw events to a data lake, aggregated results to a serving layer.
outputMode("append") is used here because we are writing individual records (no aggregation), and each record is written only once.

5.8 Step 7 – Run the Full Pipeline

5.8.1 7a – Start the Spark Job

```
spark-submit \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3 \
  etl_pipeline.py
```

spark-submit is the standard way to launch Spark applications. The `-packages` flag is equivalent to the `spark.jars.packages` config inside the code – you only need one or the other. Using `spark-submit -packages` is preferred for production as it keeps deployment config outside the source code.

5.8.2 7b – Send Messages from the Producer

In a second terminal:

```
source venv/bin/activate
python producer.py     # from TP2 solution
```

Wait for Spark's first trigger (10 seconds after start). You will see log lines like `Batch: 0`, `Batch: 1`, etc. Each batch writes Parquet files to `/tmp/spark-etl/output/`.

5.8.3 7c – Inspect the Parquet Output

After a few batches, open a Python shell and read the output:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("local[*]") \
    .getOrCreate()

# Read ALL parquet files written so far
df = spark.read.parquet("/tmp/spark-etl/output")
df.printSchema()
df.orderBy("window.start", "sensor").show(truncate=False)
```

spark.read.parquet() reads *all* Parquet files in the directory, including those from previous streaming batches. Parquet is a *columnar* format: reading only the `avg_value` column scans only that column's bytes on disk, not the entire file.
Expected output example:

window.start	sensor	avg_value
2024-01-01 10:00:00	humidity	62.40
2024-01-01 10:00:00	pressure	1008.75
2024-01-01 10:00:00	temperature	27.13

Reflection Questions

Answer these before Session 4:

1. The `value` column from Kafka arrives as **binary**. What would happen if you tried to call `.filter(col("value") > 30)` directly on the raw stream, before parsing?

2. You set `.trigger(processingTime="10 seconds")`. What trade-off does this create between latency, throughput, and the number of Parquet files generated?

3. Explain in your own words why the **watermark** is necessary for windowed aggregations. What would happen without it?

4. Your Spark job crashes after writing Batch 5. When you restart it, which batch does Spark start from, and why?

5. You want to add a new transformation to the pipeline (e.g., convert Celsius to Fahrenheit). Where in the code would you insert it, and does it require changing the checkpoint directory?

6. Summary & Key Takeaways

What We Covered Today

1. **ETL vs ELT**: ETL transforms before loading (external compute); ELT loads raw first and transforms inside the destination (cloud DW). Modern stacks prefer ELT for flexibility and reprocessability.
2. **Batch vs Streaming**: batch = high throughput, high latency, bounded data; streaming = low latency, continuous, stateful, unbounded data. The Kappa architecture unifies both with Kafka replay.
3. **Spark architecture**: Driver orchestrates, Executors compute. Transformations are lazy (DAG); actions trigger physical execution.
4. **Structured Streaming** models a stream as an infinite table. The same DataFrame API works for both batch and streaming.
5. **Watermarks** bound the lateness Spark tolerates, enabling safe finalization of time windows and freeing aggregation state.
6. **Kafka as Spark source**: `value` arrives as binary; always cast + `from_json()` before accessing fields.
7. **Checkpointing** ensures exactly-once processing guarantees and enables seamless recovery from failures.
8. In the lab, we built an end-to-end ETL pipeline: Kafka ingest → JSON parse → filter → 5-minute windowed average → Parquet sink with checkpointing.

6.1 Vocabulary Checklist

- ☑ ETL / ELT
- ☑ Schema-on-write vs schema-on-read
- ☑ Batch processing
- ☑ Streaming processing
- ☑ Lambda architecture
- ☑ Kappa architecture
- ☑ Spark Driver / Executor
- ☑ RDD / DataFrame / Dataset
- ☑ DAG (Directed Acyclic Graph)
- ☑ Lazy evaluation / action
- ☑ Structured Streaming
- ☑ Unbounded table model
- ☑ Output modes (append / update / complete)
- ☑ Watermark / late data
- ☑ Windowed aggregation
- ☑ Checkpointing
- ☑ Micro-batch / trigger
- ☑ Parquet (columnar format)

6.2 Further Reading

- Chambers, B. & Zaharia, M. (2018). *Spark: The Definitive Guide*. O'Reilly. Chapters 20–22 (Structured Streaming).
- Apache Spark Structured Streaming Guide: spark.apache.org/docs/latest/structured-streaming-programming-guide.html
- Spark + Kafka Integration: spark.apache.org/docs/latest/structured-streaming-kafka-integration.html
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly. Chapter 11 (Stream Processing).
- Flink vs Spark Streaming comparison: ververica.com/blog

6.3 Preview: Session 4

Next session we go deeper into data storage:

- Data lake concepts: raw zone, curated zone, and consumption zone.
- Storage formats in depth: Parquet, ORC, Avro – when to use each.
- Partitioning strategies for efficient querying.
- Lab: Ingest Kafka events into a structured data lake with Spark, organise data by date and sensor type, and query with Spark SQL.

Appendix A – Complete `etl_pipeline.py`

```
"""
Session 3 -- ETL Pipeline: Kafka + Spark Structured Streaming
Read sensor events from Kafka, parse JSON, filter anomalies,
compute 5-minute windowed averages, and write to Parquet.
"""
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, from_json, avg, window, to_timestamp, expr
)
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType, LongType
)

#         Config

KAFKA_BROKERS   = "localhost:9092,localhost:9094,localhost:9096"
TOPIC           = "sensor-events"
OUTPUT_PATH     = "/tmp/spark-etl/output"
CHECKPOINT_PATH = "/tmp/spark-etl/checkpoint"
TEMP_THRESHOLD  = 35.0
HUM_THRESHOLD   = 90.0

#         Spark session

spark = SparkSession.builder \
    .appName("KafkaSparkETL") \
    .master("local[*]") \
    .config(
        "spark.jars.packages",
        "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3"
    ) \
    .config("spark.sql.shuffle.partitions", "3") \
    .config("spark.streaming.stopGracefullyOnShutdown", "true") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

#         Schema

sensor_schema = StructType([
    StructField("sensor",    StringType(), False),
    StructField("value",     DoubleType(), False),
    StructField("unit",      StringType(), True),
    StructField("timestamp", LongType(),   False),
    StructField("source",    StringType(), True),
])
```

```
#         1. Read raw Kafka stream

raw = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
    .option("subscribe", TOPIC) \
    .option("startingOffsets", "earliest") \
    .option("failOnDataLoss", "false") \
    .load()

#         2. Parse JSON payload

parsed = raw \
    .select(
        col("key").cast("string").alias("message_key"),
        from_json(
            col("value").cast("string"), sensor_schema
        ).alias("data"),
        col("partition"),
        col("offset"),
    ) \
    .select(
        col("message_key"),
        col("data.sensor"),
        col("data.value"),
        col("data.unit"),
        to_timestamp(
            expr("data.timestamp / 1000")
        ).alias("event_time"),
        col("partition"),
        col("offset"),
    ) \
    .filter(col("sensor").isNotNull())

#         3. Flag anomalies

flagged = parsed.withColumn(
    "is_anomaly",
    (
        ((col("sensor") == "temperature") &
         (col("value") > TEMP_THRESHOLD))
        |
        ((col("sensor") == "humidity") &
         (col("value") > HUM_THRESHOLD))
    )
)

#         4. Windowed aggregation with watermark

windowed = flagged \
    .withWatermark("event_time", "2 minutes") \
```

```
    .groupBy(
        window(col("event_time"), "5 minutes"),
        col("sensor"),
    ) \
    .agg(avg(col("value")).alias("avg_value"))

#         5. Write aggregated stream to Parquet

query = windowed \
    .writeStream \
    .outputMode("update") \
    .format("parquet") \
    .option("path", OUTPUT_PATH) \
    .option("checkpointLocation", CHECKPOINT_PATH) \
    .trigger(processingTime="10 seconds") \
    .start()

print("Pipeline running. Ctrl+C to stop.")
query.awaitTermination()
```

Appendix B – Spark CLI Reference

```
#         Submit a PySpark application

spark-submit \
  --master local[*] \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3 \
  etl_pipeline.py

#         Launch PySpark REPL (interactive shell)

pyspark \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3

#         Read Parquet output

# (inside pyspark shell)
df = spark.read.parquet("/tmp/spark-etl/output")
df.show()
df.printSchema()

#         Count rows per window and sensor

df.groupBy("window","sensor").count().show()

#         Inspect Parquet file metadata

spark.read.parquet("/tmp/spark-etl/output").explain(True)
```

```
#         Clean checkpoint and output (start fresh)

rm -rf /tmp/spark-etl/
mkdir -p /tmp/spark-etl/{output,checkpoint}
```

Appendix C – Glossary

lightbg Term	Definition
Action	Spark operation that triggers DAG execution (e.g., `show`, `write`).
Catalyst	Spark's query optimiser that plans transformations efficiently.
Checkpoint	Persisted streaming state (offsets, watermark, aggregation) for fault recovery.
DAG	Directed Acyclic Graph of transformations built lazily before execution.
DataFrame	Distributed table abstraction with named columns and schema.
Driver	JVM process running the user's main function and scheduling tasks.
ELT	Extract-Load-Transform: load raw first, transform inside destination.
ETL	Extract-Transform-Load: transform externally before loading.
Executor	JVM process on a worker node that runs tasks and caches data.
from_json	Spark SQL function that parses a JSON string column into a struct.
Lazy evaluation	Transformations are not executed until an action is called.
Micro-batch	Small batch of records processed on each trigger interval.
Parquet	Columnar binary file format optimised for analytical queries.
RDD	Resilient Distributed Dataset: low-level Spark data abstraction.
Structured Streaming	Spark's stream processing engine built on the DataFrame API.
Trigger	Controls how often Spark runs a new streaming micro-batch.
Watermark	Threshold defining how late an event can arrive and still be processed.
Window	Time-based bucket for grouping streaming events (e.g., 5-minute tumbling).